

Informe

Desafío I - Sweet Crush | Informática II

Semestre: 2026–2

Fecha: septiembre de 2026

1. ANÁLISIS DEL PROBLEMA Y CONSIDERACIONES PARA LA ALTERNATIVA DE SOLUCIÓN PROPUESTA

1.1 Comprensión del problema

El desafío consiste en hacer una versión del juego Sweet Crush utilizando un tablero de fichas que pueda cambiar de tamaño. La principal dificultad es que cada ficha debe ocupar solamente 3 bits y estos deben estar guardados de manera seguida dentro de un arreglo de bytes.

Esto complica bastante el manejo del tablero, porque un byte tiene 8 bits y una ficha tiene 3. Por ejemplo, algunas fichas quedan completamente dentro de un byte, pero otras empiezan en un byte y terminan en el siguiente. Por eso no se puede simplemente acceder a cada ficha como si ocupara un espacio completo del arreglo.

Además, el ejercicio tiene varias restricciones que afectan la forma de hacer el programa:

- No se pueden usar `struct`, `class`, `template`, `STL` ni objetos definidos por el estudiante.
- No se puede utilizar `std::string`.
- Se deben utilizar punteros, arreglos y memoria dinámica.
- El programa debe estar separado en archivos `.h` y `.cpp`.
- Las operaciones a nivel de bits son necesarias para poder guardar y modificar las fichas.
- Por estas condiciones, una buena parte del trabajo estuvo en encontrar una forma de manejar el tablero sin perder la información de las fichas y sin utilizar estructuras.

1.2 Decisiones de diseño y alternativas consideradas

Durante el desarrollo se fueron tomando algunas decisiones según las restricciones del ejercicio y según lo que resultaba más fácil de manejar.

Codificación de las fichas.

Se utilizaron los códigos 000 hasta 101 para los seis tipos de fichas. El 110 representa una posición vacía y 111 queda reservado.

Al principio se podría pensar en usar 2 bits por ficha, pero esto solamente permitiría seis valores diferentes. Como también necesitamos representar una posición vacía, se decidió utilizar los 3 bits completos. De esta forma todas las posiciones tienen el mismo tamaño y el manejo del tablero es más sencillo.

Tipo de buffer.

El tablero se guarda en un arreglo dinámico de `unsigned char`. No se utilizó un `int` para cada ficha porque se desperdiciaría mucho espacio. Además, hacerlo de esa forma no cumpliría con la condición de que las fichas estén almacenadas en 3 bits.

Paso del estado del tablero.

Normalmente se podría utilizar un `struct` para guardar el puntero al tablero, las filas, las columnas y la capacidad. Como esto está prohibido, esos datos se pasan directamente a las funciones.

Esto hace que algunas funciones tengan varios parámetros, pero es una forma de mantener toda la información necesaria sin utilizar una estructura.

Manejo de los bits sobrantes.

Cuando la cantidad de fichas no ocupa exactamente todos los bits del último byte, quedan algunos bits sin utilizar. Estos se mantienen juntos al final del buffer.

No se puede dejar un espacio entre cada ficha porque eso cambiaría la forma en que están almacenadas y haría que ya no fueran exactamente 3 bits por ficha.

Arreglo temporal para las combinaciones.

Para saber qué fichas se deben eliminar se utiliza un arreglo temporal de `bool`.

Este arreglo no almacena las fichas del tablero. Solamente indica qué posiciones deben eliminarse. Por eso no es necesario guardarlo utilizando 3 bits.

Además, esto ayuda cuando una ficha forma parte de una combinación horizontal y otra vertical al mismo tiempo. La posición simplemente queda marcada y se elimina una sola vez.

Generación de fichas aleatorias.

Para generar las fichas se utilizaron `rand()` y `srand()` de `<cstdlib>`. Se decidió no utilizar `std::mt19937` porque implicaría trabajar con un objeto y el ejercicio tiene restricciones con respecto al uso de objetos de bibliotecas.

Detección horizontal y vertical.

Se hicieron dos funciones diferentes: `Buscar_horizontal` y `Buscar_vertical`.

También se podría haber hecho una sola función que recibiera un parámetro para indicar si debía buscar horizontal o verticalmente. Sin embargo, eso habría agregado más condiciones dentro de la función. Se prefirió separar las dos tareas para que cada función hiciera una sola cosa.

Eliminación de columnas.

Para `Eliminar_columna` y `Eliminar_Fila` se utiliza un arreglo temporal. El problema es que cuando se elimina una columna o fila, las columnas o filas que están después cambian de posición.

Si se modifica directamente el mismo buffer, existe el riesgo de sobrescribir una ficha antes de haberla copiado. Con el arreglo temporal primero se guardan los datos y después se construye el nuevo orden.

Símbolos de las fichas.

Para mostrar las fichas en pantalla se utiliza `char`. En este caso no se necesita almacenar información adicional, solamente un carácter que permita identificar visualmente cada tipo de ficha.

1.3 Arquitectura general

El programa se dividió en cinco módulos para no tener todo el código mezclado.

El módulo de memoria se encarga de reservar, liberar y cambiar el tamaño del buffer cuando es necesario.

El módulo bitwise contiene las funciones que trabajan directamente con los bits. Aquí están principalmente `Extraer_ficha` y `Escribir_ficha`.

El módulo de interfaz se encarga de mostrar el tablero. También permite mostrar la información en binario para comprobar que las fichas realmente estén almacenadas de la forma esperada.

El módulo de lógica contiene las funciones principales del juego, como buscar combinaciones, eliminar fichas, aplicar gravedad, rellenar espacios y manejar las cascadas.

Por último, el módulo de estadísticas se encarga de guardar los datos relacionados con las eliminaciones, combinaciones, cascadas y puntuación.

Se decidió que los otros módulos no trabajaran directamente con los bits del buffer. Cuando necesitan conocer o modificar una ficha utilizan `Extraer_ficha` y `Escribir_ficha`. De esta manera, si se cambia la forma de guardar una ficha, no habría que modificar todo el programa.

2. ESQUEMA DE TAREAS PARA EL DESARROLLO DE LOS ALGORITMOS

Bloque 1 — Base: primero se trabajó en cómo guardar las fichas usando 3 bits y en calcular cuántos bytes eran necesarios para el tablero. Después se hizo la parte de memoria dinámica.

Bloque 2 — Acceso y visualización: después se hicieron `Extraer_ficha` y `Escribir_ficha`. Esta parte fue importante porque había que comprobar que funcionaran tanto las fichas que cabían en un byte como las que quedaban divididas entre dos. También se agregó la generación aleatoria y la visualización.

Bloque 3 — Lógica del juego: después se implementó la búsqueda de combinaciones horizontales y verticales, la eliminación, la gravedad, el relleno y las cascadas.

Bloque 4 — Cambios en el tablero y finalización: finalmente se trabajó en agregar y eliminar filas y columnas, incluyendo posiciones intermedias. También se agregaron las estadísticas, la puntuación y las pruebas necesarias para la demostración.

3. ALGORITMOS IMPLEMENTADOS

3.1 Extracción y escritura de fichas

Esta fue una de las partes más importantes del programa. El problema es que una ficha ocupa 3 bits y un byte tiene 8 bits, así que las fichas no siempre están alineadas.

Para saber dónde empieza una ficha se calcula:

```
bit_inicial = indice * 3
```

Después se obtiene el byte donde empieza:

```
byte = bit_inicial / 8
```

y la posición dentro del byte:

```
desplazamiento = bit_inicial % 8
```

Con estos valores se utilizan desplazamientos y máscaras para obtener los 3 bits.

Si los 3 bits están dentro del mismo byte, el proceso es directo. Si la ficha queda dividida entre dos bytes, se toman los bits correspondientes de cada uno y se combinan.

La misma idea se utiliza para escribir una ficha. Primero se limpian los bits que se van a modificar y después se coloca el nuevo valor.

3.2 Detección de combinaciones

Para buscar combinaciones se utilizan `Buscar_horizontal` y `Buscar_vertical`.

En la búsqueda horizontal se recorre cada fila y se comparan las fichas que están una al lado de la otra. Cuando se encuentran tres o más iguales, se marcan.

En la búsqueda vertical se hace algo parecido, pero recorriendo cada columna.

El arreglo temporal de `bool` sirve para guardar las posiciones que se deben eliminar. Una ventaja de hacerlo así es que si una ficha está en una combinación horizontal y vertical, la misma posición puede quedar marcada sin necesidad de eliminarla dos veces.

3.3 Gravedad y relleno

Después de eliminar las fichas quedan espacios vacíos. Para solucionarlo se aplica la gravedad.

Se recorre cada columna desde abajo y se utilizan dos posiciones: una indica dónde se debe colocar la siguiente ficha y la otra busca una ficha que todavía se pueda mover.

Cuando se encuentra una ficha, se mueve hacia abajo.

Cuando termina este proceso, los espacios que quedaron arriba se rellenan con nuevas fichas aleatorias.

En esta parte se utilizó `while` porque el proceso depende de la cantidad de espacios vacíos y de las fichas que se encuentren. No siempre se hacen exactamente los mismos movimientos.

3.4 Cascadas

Una eliminación puede provocar otra combinación. Por eso después de cada eliminación se vuelve a revisar el tablero.

El proceso es:

1. Buscar combinaciones.
2. Eliminar las fichas encontradas.
3. Aplicar gravedad.
4. Rellenar los espacios.
5. Volver a buscar combinaciones.

Esto continúa hasta que ya no haya ninguna combinación.

3.5 Modificación estructural

Para agregar una fila o una columna se crea un nuevo buffer con el tamaño necesario. Luego se copian las fichas del tablero anterior en las posiciones correspondientes, teniendo en cuenta dónde se insertó la nueva fila o columna.

Para eliminar se hace algo parecido, pero simplemente no se copia la fila o columna que se quiere quitar.

La memoria solamente se reduce cuando el uso del buffer baja del 65 %, tal como lo pide el ejercicio.

3.6 Puntuación

Para la puntuación se establecieron 10 puntos por cada ficha eliminada.

Cuando hay una cascada, las rondas posteriores aumentan la puntuación utilizando un factor creciente. De esta forma una combinación que forma parte de una cascada puede dar más puntos que una eliminación normal.

4. PROBLEMAS DE DESARROLLO AFRONTADOS

Uno de los primeros problemas fue entender cómo manejar las fichas que quedan repartidas entre dos bytes. Al principio es fácil pensar que cada ficha puede leerse directamente de un byte, pero esto no siempre funciona. Fue necesario utilizar desplazamientos y máscaras para poder manejar los dos casos.

Otro problema apareció al agregar y eliminar filas y columnas. Estos cambios hacen que cambien las posiciones de muchas fichas, por lo que no era suficiente con modificar algunos índices. Fue necesario crear un buffer nuevo, copiar la información y liberar el buffer anterior.

Con las cascadas también hubo problemas. Una combinación puede generar otra después de aplicar gravedad y rellenar el tablero. Por eso había que asegurarse de que el programa siguiera buscando mientras aparecieran nuevas combinaciones, pero que también pudiera detenerse cuando ya no hubiera ninguna.

Otro caso que hubo que tener en cuenta fue una ficha que pertenece a dos combinaciones al mismo tiempo. Para esto se utilizó el arreglo temporal de `bool`, que permite marcar una posición una sola vez.

5. EVOLUCIÓN DE LA SOLUCIÓN Y CONSIDERACIONES PARA LA IMPLEMENTACIÓN

5.1 Evolución

El programa se fue construyendo por partes.

Primero se definió cómo representar una ficha con 3 bits y cómo calcular el tamaño del buffer.

Después se trabajó en `Extraer_ficha` y `Escribir_ficha`. En esta etapa se hicieron pruebas para comprobar que funcionaran las fichas que estaban dentro de un solo byte y las que estaban divididas entre dos.

Luego se empezó con la lógica del juego. Primero se hicieron las búsquedas horizontal y vertical y después la eliminación, la gravedad y el relleno. Una vez que estas partes funcionaban se agregaron las cascadas.

Después se trabajó en los cambios de tamaño del tablero. Se implementó la posibilidad de agregar y eliminar filas y columnas, incluso cuando la posición no estaba al principio o al final.

Finalmente se agregaron las estadísticas, la puntuación y la visualización.

En las últimas pruebas se verificaron las situaciones necesarias para la demostración: una combinación horizontal, una vertical, cambios de filas y columnas en posiciones intermedias y una cascada de tres o más rondas.

5.2 Consideraciones para la implementación

El tablero siempre debe mantenerse utilizando la representación de 3 bits por ficha. Los arreglos auxiliares solamente se utilizan durante procesos específicos y no reemplazan la forma principal de almacenar el tablero.

Los bits que no se utilicen deben quedar agrupados al final del buffer y no entre las fichas.

La memoria se debe reducir solamente cuando el porcentaje de uso sea menor al 65 %. También es importante que la lógica del juego no manipule directamente los bits. Para eso se utilizan `Extraer_ficha` y `Escribir_ficha`, que concentran esa parte del programa.

La puntuación debe mantenerse definida de la misma forma tanto en el código como en el informe.

Por último, se deben mantener los archivos organizados y realizar commits durante el desarrollo. Esto permite mostrar cómo fue avanzando el programa y qué cambios se fueron haciendo.